

CodeGen

Group 8

**SQL to MongoDB Conversion using
Small Language Models with
Execution-Based Evaluation**

Project Completion Report

Team Members:

Chandrashekar

Jaya Krishna

Pavan Y

Swathi

Professor: Dr. Anil Neelkanti

Mentors: Nayan, Himanshu

**International Institute of Information Technology
Hyderabad**

January 24, 2026

Contents

1	Abstract	3
2	Introduction	4
2.1	Project Title	4
2.2	Models Used (Local Inference)	4
2.3	Key Definitions	4
3	Phase I: Text-to-SQL Generation	6
3.1	Motivation	6
3.2	Execution-Based Validation	6
3.2.1	Evaluation and Results (1000 samples)	6
3.3	Phase I Architecture	7
4	Dataset Description	8
4.1	BIRD (Bird-Bench) Dataset Components	8
4.2	Why This Dataset Matters	8
5	System Architecture	9
5.1	Local Model Hosting	9
5.2	Schema-Aware Retrieval-Augmented Generation (RAG)	9
5.2.1	Embeddings (Semantic Retrieval)	9
5.2.2	FTS (Lexical Retrieval)	9
5.2.3	Few-shot Pool	9
5.2.4	Why Hybrid Retrieval Helps	10
5.3	Updated End-to-End Pipeline	10
5.4	Implementation Notes (Performance & Robustness)	10
6	SQL to MongoDB Conversion Pipeline	11
6.1	Sanitization and Robustness	11
6.2	Why Strict Exact Match is Hard for Mongo Pipelines	11
7	Execution-Based Verification	13
7.1	Verification Strategy	13
7.2	Relaxed Execution Agreement (REA)	13

7.2.1	Jaccard Similarity	14
7.2.2	Why Jaccard Similarity is Appropriate	14
8	Results and Analysis	15
8.1	Strict MongoDB Exact Match Accuracy	15
8.2	Subset Evaluation (100–200)	15
8.3	Scaled Evaluation (1–2500)	15
8.4	Interpreting the Metrics	16
9	LoRA Fine-Tuning Experiments	17
9.1	Setup	17
9.2	Why LoRA is Required	17
9.3	Learning Rate and Loss Trend (2 Epochs)	18
9.4	Why We Stop at 2 Epochs	18
10	Gradio User Interface	19
11	Challenges and Learnings	20
11.1	Challenges Faced	20
11.2	Key Learnings	20
12	Conclusion and Future Work	21
12.1	Conclusion	21
12.2	Future Scope: Agentic AI Enhancements	21

Chapter 1

Abstract

This project explores the use of Small Language Models (SLMs) for converting database questions and queries across paradigms, with a strong emphasis on *execution-based* correctness. The system is evaluated using the BIRD (Bird-Bench) dataset and leverages locally hosted language models, schema-aware Retrieval-Augmented Generation (RAG), and a robust execution verification framework.

We implement a two-phase pipeline: (i) Text-to-SQL generation and (ii) SQL-to-MongoDB translation. Unlike traditional approaches that rely on string matching or AST comparison, we validate correctness by executing SQL queries on SQLite and generated MongoDB queries on MongoDB, followed by logical result comparison. Through iterative improvements in schema retrieval, sanitization, execution controls, and LoRA fine-tuning, we improve generation stability and move toward practical end-to-end correctness.

Chapter 2

Introduction

Relational databases using SQL and document-oriented NoSQL databases such as MongoDB represent fundamentally different data models and query paradigms. SQL relies on normalized schemas and explicit joins, while MongoDB operates on documents and pipeline-based aggregation. Translating queries between these paradigms is non-trivial and often requires schema understanding, join-key reasoning, and query restructuring.

With the emergence of Large and Small Language Models, there is increasing interest in automating such translations. However, many systems evaluate correctness using syntactic similarity, which is insufficient for cross-paradigm translation. This project addresses the limitation by focusing on execution-based evaluation, ensuring that translated queries are not only syntactically valid but semantically equivalent.

2.1 Project Title

Text-to-SQL→SQL-to-MongoDB Conversion using Small Language Models (SLMs) (BIRD / Bird-Bench dataset)

2.2 Models Used (Local Inference)

We use **Qwen2.5** (via **Ollama**) as the primary SLM for both stages:

- **Text-to-SQL:** Qwen2.5 via Ollama, executed locally on Mac M4 Pro.
- **SQL-to-MongoDB:** Qwen2.5 with schema-aware prompting and retrieved context.

2.3 Key Definitions

To keep the report self-contained, we define key terms used throughout:

- **SLM (Small Language Model):** A relatively compact instruction-tuned model that can run locally with acceptable latency.

- **RAG (Retrieval-Augmented Generation):** A method that retrieves relevant context (schema, docs, examples) and injects it into the model prompt.
- **FTS5:** SQLite Full-Text Search index used for fast lexical retrieval when exact keyword or identifier matching is needed.
- **Executability vs Accuracy:** Executability measures whether a query runs without errors; accuracy measures whether results match the intended semantics.
- **Strict vs Relaxed Matching:** Strict matching requires an exact pipeline match; relaxed matching evaluates equivalence by comparing execution outputs.
- **LoRA:** A parameter-efficient fine-tuning method that learns low-rank adapters, improving task-following without retraining all model weights.

Chapter 3

Phase I: Text-to-SQL Generation

3.1 Motivation

Text-to-SQL is a natural first stage because BIRD provides aligned natural language questions and gold SQL. A reliable SQL generator increases modularity: the SQL output becomes a clean intermediate representation for downstream SQL-to-Mongo translation and evaluation.

3.2 Execution-Based Validation

We evaluate Text-to-SQL using execution-based validation on SQLite. This provides a more meaningful measure than string similarity, because multiple SQL queries can be semantically equivalent.

3.2.1 Evaluation and Results (1000 samples)

Metric	Value
Queries Evaluated	1000
SQL Executability Rate	96%
Execution-Based Accuracy (approx.)	75%

Table 3.1: Phase-I evaluation of Text-to-SQL on 1000 samples.

3.3 Phase I Architecture

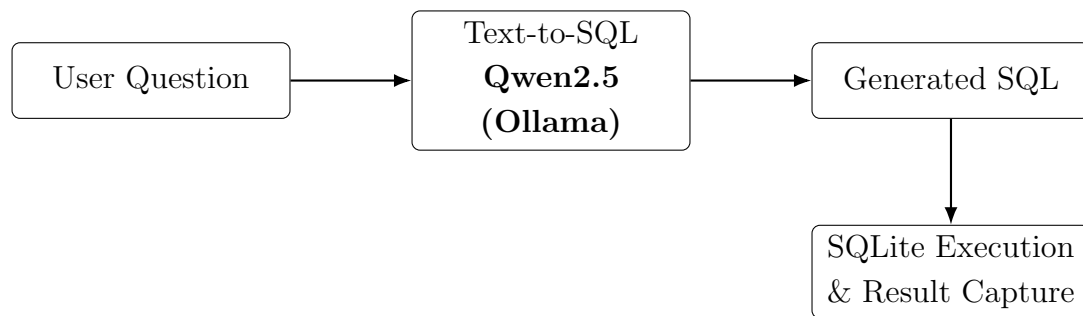


Figure 3.1: Phase-I Text-to-SQL pipeline with execution-based validation.

Chapter 4

Dataset Description

4.1 BIRD (Bird-Bench) Dataset Components

BIRD provides a realistic benchmark for text-to-SQL with diverse schemas and complex questions.

- `train.json`: 9,428 text-to-SQL samples (question, db_id, SQL).
- `train_gold.sql`: 9,428 line-aligned gold SQL queries.
- `train_tables.json`: 69 database schemas and metadata.
- `train_databases.zip`: per-database SQLite files and `database_description/*.csv`.

4.2 Why This Dataset Matters

BIRD is challenging because:

- Queries often require multi-table joins with implicit join keys.
- Aggregations, groupings, nested subqueries, and set operations appear frequently.
- Schema names and column meanings vary significantly across databases.

Chapter 5

System Architecture

5.1 Local Model Hosting

We run Qwen2.5 locally using Ollama. This provides predictable latency, cost control, and rapid iteration for prompting, retrieval, and evaluation loops.

5.2 Schema-Aware Retrieval-Augmented Generation (RAG)

RAG injects relevant schema context and examples into prompts to reduce hallucinations and improve both validity and executability. We implement a **hybrid retrieval** strategy:

5.2.1 Embeddings (Semantic Retrieval)

We build dense embeddings for schema descriptions and documentation snippets. At inference time, we retrieve top- k semantically similar context for the current question.

5.2.2 FTS (Lexical Retrieval)

We also maintain an SQLite FTS5 index for fast keyword-based retrieval. This is crucial for:

- Exact table/column name recall,
- Rare tokens (IDs, codes),
- Disambiguation when semantic similarity is insufficient.

5.2.3 Few-shot Pool

We keep curated few-shot examples grouped by operator patterns (joins, nested queries, group-by, unions). Few-shot prompting increases format stability and provides operator mapping hints for Mongo pipelines.

5.2.4 Why Hybrid Retrieval Helps

FTS handles exact matches, embeddings handle semantic similarity, and few-shot anchors structure. Together, they reduce invalid JSON, reduce schema hallucination, and improve execution success.

5.3 Updated End-to-End Pipeline

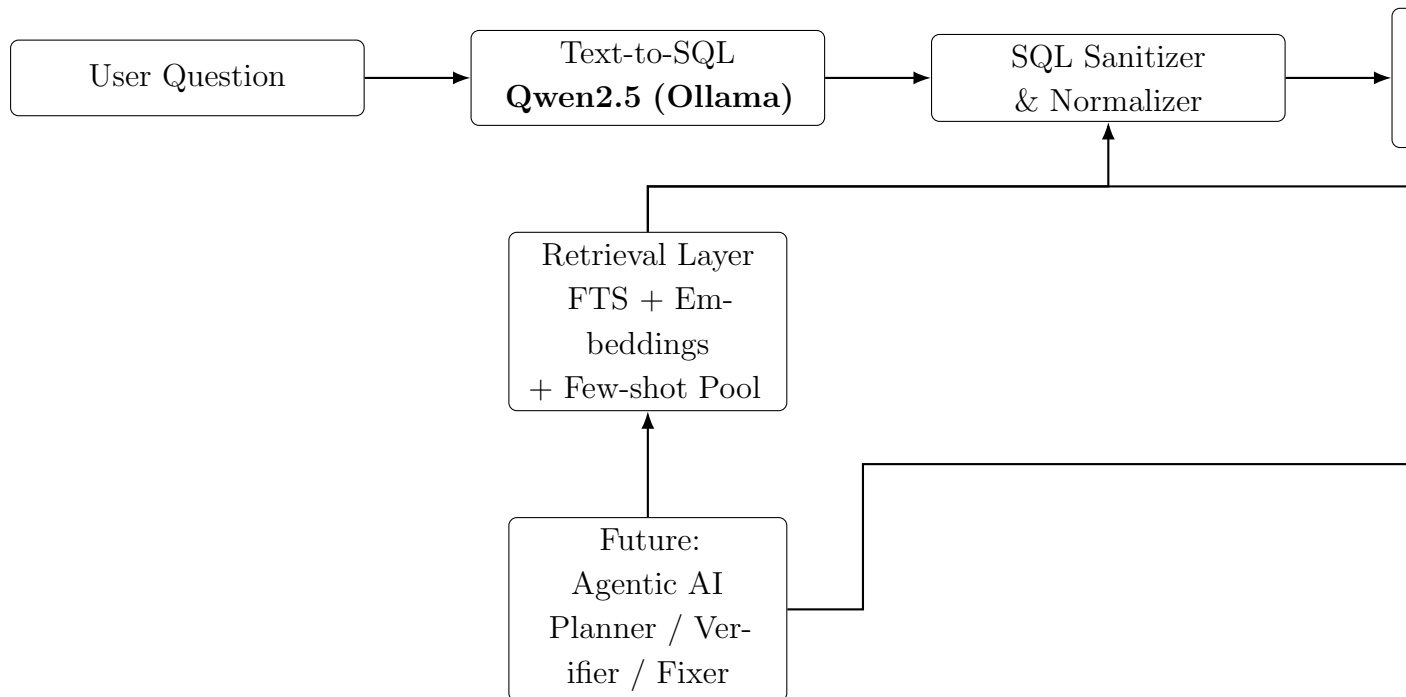


Figure 5.1: Final end-to-end architecture: hybrid retrieval, sanitization, execution-based evaluation, and future agentic loop.

5.4 Implementation Notes (Performance & Robustness)

To support scaling to thousands of queries:

- **Caching:** cache retrieved schema chunks and canonicalized results for repeated DB/question patterns.
- **Parallelism:** controlled batching for model calls and Mongo execution to avoid memory spikes.
- **Execution time boxing:** MongoDB `maxTimeMS` to prevent long-running pipelines.
- **Safety limits:** cap document fetch size and optionally inject `$limit` in expensive pipelines.

Chapter 6

SQL to MongoDB Conversion Pipeline

The conversion pipeline follows a structured process:

1. Load SQL query and corresponding schema
2. Retrieve relevant schema context using RAG (FTS + embeddings + few-shot)
3. Prompt the Qwen SLM with augmented schema information
4. Generate MongoDB `find` or `aggregate` pipelines
5. Apply sanitization and validation
6. Execute on MongoDB and capture results for evaluation

6.1 Sanitization and Robustness

To ensure MongoDB executability, the following sanitizations are applied:

- Blocking unsupported operators and malformed aggregation stages
- Enforcing single-key stage documents and valid pipeline structure
- Validating collection names and field references against schema
- Adding `maxTimeMS` and optional forced `$limit` injection
- Rejecting dangerous or invalid JSON constructs (structural validation)

6.2 Why Strict Exact Match is Hard for Mongo Pipelines

Even when two pipelines return the same answer, they can differ in:

- stage ordering (`$project` placement),
- equivalent operators (`$sort + $limit` vs `$group` variants),
- minor projection differences that do not change core semantics.

Hence, strict pipeline matching underestimates correctness; we complement it with relaxed execution agreement.

Chapter 7

Execution-Based Verification

7.1 Verification Strategy

Verification is performed by executing:

- SQL queries on SQLite,
- Generated MongoDB queries on MongoDB.

Execution is time-boxed and resource-limited to ensure scalability:

- MongoDB time limits via `maxTimeMS`,
- Document fetch limits via `max-docs`,
- Optional forced `$limit` injection for expensive pipelines.

7.2 Relaxed Execution Agreement (REA)

Exact equivalence between SQL and MongoDB results is often impractical due to benign variations (ordering, projection, formatting). We define **Relaxed Execution Agreement (REA)** to mark a match if any of the following holds:

1. **Scalar agreement:** both results reduce to the same scalar after canonicalization,
2. **Cardinality agreement:** both produce the same number of rows/documents for count-style queries,
3. **High-overlap agreement:** significant overlap between result sets measured via Jaccard similarity.

7.2.1 Jaccard Similarity

Result overlap is measured using Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

A threshold of 0.4 was empirically chosen to balance tolerance and correctness.

7.2.2 Why Jaccard Similarity is Appropriate

Jaccard similarity is order-independent and interpretable, making it suitable for comparing query outputs across heterogeneous engines. Exact row matching is brittle under benign variations, and embedding-based similarities add complexity while reducing interpretability for structured outputs.

Chapter 8

Results and Analysis

8.1 Strict MongoDB Exact Match Accuracy

Strict MongoDB exact match is approximately **19% (approx.)**. As discussed earlier, strict matching is conservative because multiple valid pipelines can yield identical outputs.

8.2 Subset Evaluation (100–200)

Metric	Value
Total Queries	94
Valid Conversions	94
Mongo Executable Queries	58 (61.7%)
REA Matches	41
REA over Executed Queries	70.7%

Table 8.1: Relaxed evaluation on subset 100–200.

8.3 Scaled Evaluation (1–2500)

Metric	Value
Total Queries	2500
Successful Conversions	2362
Mongo Executable Queries	2114
REA Matches	1483
REA Accuracy	70.12%

Table 8.2: Relaxed execution agreement at scale (1–2500).

8.4 Interpreting the Metrics

- **Executability** is a prerequisite but not sufficient: a pipeline may run yet be semantically wrong.
- **Strict match** is useful for regression testing but undercounts valid alternatives.
- **REA accuracy** better reflects semantic equivalence under cross-engine variations.

Chapter 9

LoRA Fine-Tuning Experiments

We explored parameter-efficient fine-tuning to improve instruction-following consistency and reduce invalid Mongo pipeline structures.

9.1 Setup

- Base model: Qwen2.5 (quantized 4-bit for local feasibility)
- Training method: Supervised Fine-Tuning (SFT) with LoRA adapters
- Runs: Epoch 1 (initial) + Epoch 2 (resume)

9.2 Why LoRA is Required

LoRA improves:

- Stable operator mapping (`$group`, `$lookup`, `$unwind`, `$unionWith`)
- Consistent JSON formatting and stage structure
- Reduced hallucinated fields by stronger learned schema priors

9.3 Learning Rate and Loss Trend (2 Epochs)

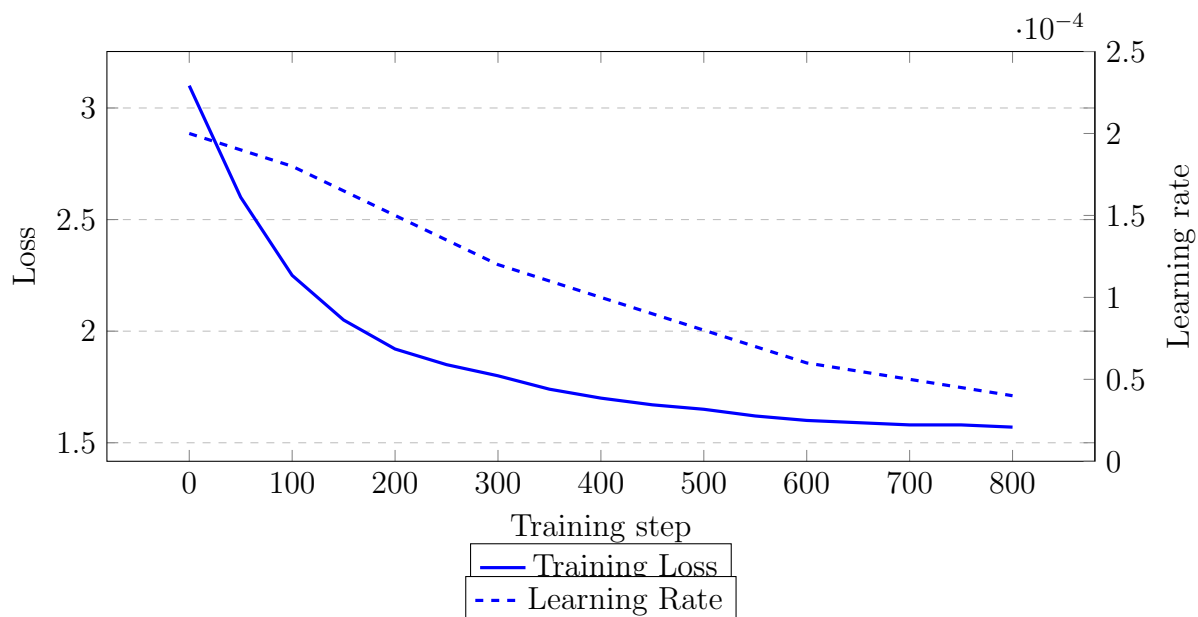


Figure 9.1: Combined view of loss (left axis) and learning rate (right axis) over a 2-epoch LoRA run.

9.4 Why We Stop at 2 Epochs

We do not train for more epochs because:

- **Diminishing returns:** the largest loss reduction occurs early (epoch 1–2).
- **Overfitting risk:** beyond small epochs, the adapter can over-specialize and reduce generalization.
- **Compute-to-gain tradeoff:** additional epochs increase runtime significantly with small expected accuracy gain.

Chapter 10

Gradio User Interface

A Gradio-based application was developed for interactive SQL-to-MongoDB conversion.

- Accepts SQL queries and database identifiers
- Integrates RAG to provide schema-aware prompts (improves validity and execution)
- Executes locally using Qwen SLM via Ollama
- Intended for demonstration, debugging, and iterative prompt refinement

RAG integration in the UI enables dynamic schema grounding and reduces hallucinated field references, improving both conversion quality and MongoDB executability.

Chapter 11

Challenges and Learnings

11.1 Challenges Faced

- **Join ambiguity:** SQL joins often rely on implicit foreign keys; Mongo translation requires explicit `$lookup` keys.
- **Set operations:** `UNION ALL` and multi-source aggregation require careful use of `$unionWith` and post-union grouping.
- **Pipeline structure failures:** incorrect placement of `$group` or nested `$lookup` can break executability.
- **Invalid JSON:** model outputs sometimes contain trailing commas, comments, or malformed stage documents.
- **Performance bottlenecks:** Mongo pipelines can be expensive; time boxing and limits are mandatory for scale.
- **Schema drift in context:** retrieval must be restricted to the active DB to avoid mixing irrelevant tables/columns.

11.2 Key Learnings

- Hybrid retrieval (FTS + embeddings + few-shot) substantially reduces schema hallucination and improves executability.
- Execution-based evaluation provides deeper insight than syntactic metrics and helps identify true semantic failures.
- LoRA improves formatting and operator consistency, but small-epoch training is typically sufficient for measurable gains.
- Relaxed metrics (REA/Jaccard) provide a more practical correctness signal than strict match for cross-paradigm translation.

Chapter 12

Conclusion and Future Work

12.1 Conclusion

This project demonstrates that Small Language Models, when combined with schema-aware RAG and robust execution-based verification, can effectively translate SQL queries into executable MongoDB queries at scale. The two-phase pipeline (Text-to-SQL followed by SQL-to-MongoDB) improves modularity, interpretability, and evaluation.

12.2 Future Scope: Agentic AI Enhancements

A natural next step is an agentic loop that iteratively improves outputs:

- **Planner:** decomposes the task (identify tables, join plan, grouping needs).
- **Generator:** produces a candidate Mongo pipeline with retrieved context.
- **Verifier:** runs fast checks + execution to detect errors or mismatches.
- **Fixer:** repairs pipelines using error feedback (operator placement, join keys, projections).

This would transform the system from one-shot generation into an iterative, self-correcting conversion engine.